Suraksha: HPC-based backdoor screening

Overview

For this submission, I treated the problem as clean-only anomaly detection. The training file has normal inference runs but no labelled backdoor runs, so there is no attack class to train a conventional classifier on. Instead, Suraksha builds a profile of the clean measurements and flags rows that are unusually far from that profile.

The training file contains 800 rows and three hardware performance counters: `cache-references`, `cycles`, and `LLC-loads`. The checked-in model uses an alert threshold of `19.742107`, the 99.5th percentile of the scores on those clean rows.

Why use hardware counters?

Hardware performance counters measure what the processor is doing while an inference runs. Cache activity and cycle counts can change when code takes an extra branch or performs additional work. A backdoor may therefore leave a hardware-level trace even when the model returns an ordinary-looking result.

That signal is not proof of a backdoor. Operating-system activity, input difficulty, hardware differences, and measurement noise can all move the counters. The detector should be read as a screening step for unusual executions.

Scoring method

The program in `detector.py` uses only Python's standard library:

1. Read the three numeric counter columns from the clean CSV.
2. Use the median of each column as its clean baseline.
3. Use `1.4826 × MAD` (median absolute deviation) as the spread. If a column is flat, fall back to its population standard deviation and then to `1.0`.
4. For every row, add the squared standardized distance for each counter:

```
score = sum(((value - median) / scale) 2)
```

1. **Mark a row as `backdoor` only when its score is strictly greater than the 99.5th-percentile threshold. Otherwise it is `clean`.**

Using the median and MAD makes the baseline less sensitive to a few unusual clean measurements than a mean and standard deviation would be. Squaring and adding the normalized distances also puts the three counters on a common scale.

Reproducing the result

From the repository root:

`bash python3 detector.py train \ --input clean_training.csv \ --model suraksha_model.json

````
To score another file:

```bash
python3 detector.py predict \
  --input validation.csv \
  --model suraksha_model.json \
  --output predictions.csv
````

The input file must use the same three column names. The output records the source row number, the anomaly score, and the prediction.

Limitations

The clean file alone cannot measure recall, F1, AUROC, or the true false-positive rate. Those numbers require labelled evaluation data. A high score can mean an unusual but legitimate execution, not necessarily an attack.

The current implementation scores rows independently. It does not model correlations between counters, sequences over time, input metadata, or repeated measurements of the same inference. It also assumes that future clean runs look reasonably similar to the training data. If labelled validation data becomes available, it should be used to choose the threshold and measure the detector against the required metric.
````